



Amrita School of AI
COIMBATORE CAMPUS

RELEASED WEEK 3 · DUE WEEK 5

Assignment 1: Measure Before You Optimise

Take one slow program, find out what is actually limiting it, and prove your answer with a roofline calculation before you change a line.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

RELEASED

Week 3

DUE

End of week 5

WEIGHT

1.5% of the course (assignments total 6%)

MARKS

30

WORK

Individual

OFFERING

Odd Semester 2026-27

What this is for

Almost every student, given a slow program, starts changing it. This assignment forbids that until you can say what the ceiling is.

You will take one kernel, characterise the machine it runs on, compute its arithmetic intensity by hand, place it on a roofline, predict its maximum attainable performance, and only then measure. The mark is for the reasoning, not for how fast you make it go.

The kernel

A 5-point Jacobi stencil on a 2-D grid, the same one that reappears in Unit 2 as an MPI exercise:

```
for (int step = 0; step < steps; step++) {  
    for (int i = 1; i < n - 1; i++)  
        for (int j = 1; j < n - 1; j++)  
            b[i][j] = 0.25 * (a[i-1][j] + a[i+1][j] + a[i][j-1] + a[i][j+1]);  
    swap(a, b);  
}
```

Write it, in C, serial, correct. Use `double`. Take `n` and `steps` from the command line. Verify it against a known result before timing anything: an incorrect kernel that is fast is worth nothing.

Part A: Characterise the machine (6 marks)

Establish, for the machine you will use, and **state how you obtained each number**:

QUANTITY	HOW TO GET IT
Cores (physical, not logical)	<code>lscpu</code> , and say which is which
Clock frequency under sustained load	measure it, do not quote the label
L1, L2, L3 sizes	<code>lscpu</code> , confirmed by your Lab 1 staircase
Peak FP64 rate	cores × clock × FLOP per cycle; state your assumption
Achievable memory bandwidth	measure it with a STREAM-like triad, do not quote the datasheet

PITFALL · PEAK FROM THE DATASHEET IS NOT THE PEAK YOU GET

Sustained clocks are lower than boost clocks, and achievable bandwidth is typically 75 to 85 percent of the theoretical figure. Report measured numbers and say how you measured them. A number you cannot justify earns nothing.

Compute and report the **ridge point** of your machine in FLOP per byte.

Part B: Predict, then measure (10 marks)

B1. Count, by hand, for one interior point of the stencil:

- floating-point operations
- bytes that must come from DRAM, *assuming perfect cache reuse of the rows already loaded*
- bytes that must come from DRAM *assuming no reuse at all*

These two assumptions bracket the truth. Report both intensities.

B2. Place both bracketing points on your roofline. State the attainable GFLOP/s in each case. Is this kernel memory bound or compute bound on your machine? Say which, and by how much.

B3. Now measure. Run at $n = 256, 1024, 4096$ with enough steps that each run takes at least half a second. Report achieved GFLOP/s for each, using the timing discipline from Lab 1: warm-up, at least five runs, report the minimum and the spread.

B4. Compare prediction with measurement. Where does the measured point fall relative to your two brackets? If it is outside them, your model is wrong somewhere. Say where.

NOTE · BEING WRONG IS ALLOWED. NOT NOTICING IS NOT.

A prediction that misses, followed by a correct diagnosis of why, scores full marks. A prediction that happens to match, with no reasoning shown, does not.

Part C: Optimise, with justification (10 marks)

Now you may change the code. For **each** change you make:

1. State what you expected to happen, and why, *before* measuring.
2. Measure.
3. Report whether the prediction held.

Apply at least three of:

- loop order and access pattern
- blocking or tiling for cache
- avoiding the array copy in the swap (pointer swap instead)
- compiler flags, including `-march=native`
- eliminating redundant loads by keeping row pointers
- forcing or preventing vectorisation, and checking the compiler's report

PITFALL · ONE CHANGE AT A TIME

Change one thing, measure, record. Changing three things and reporting the total is not an experiment, and you will not be able to say which of them mattered.

Report a table: version, time, GFLOP/s, percentage of the attainable ceiling from Part B, and your prediction versus the outcome.

Part D: Know when to stop (4 marks)

D1. State your final position as a percentage of the attainable roofline ceiling, not as a percentage of peak.

D2. Answer, with evidence: is further optimisation of this kernel worth doing? If yes, name the specific next step and the expected gain. If no, say what would have to change, in the algorithm or the hardware, before more effort paid off.

D3. One paragraph: what would you have done differently if you had started optimising immediately instead of computing the ceiling first?

What to submit

A single PDF, `assign1-<ROLLNO>.pdf`, plus your source as a separate `.c` file. Name both for your roll number.

The report must contain:

- your machine characterisation table with methods
- your roofline plot, hand-drawn or generated, with your kernel's bracketed positions and every measured point marked
- the prediction-versus-measurement comparison
- the optimisation table
- your stopping argument

MARKS	FOR
6	Part A: measured, justified machine characterisation, correct ridge point
10	Part B: correct hand-computed intensity, honest bracketing, and a real reconciliation with measurement
10	Part C: at least three changes, each with a stated prediction and an isolated measurement
4	Part D: a defensible stopping argument grounded in the roofline

NOTE · REPRODUCIBILITY IS PART OF THE MARK, NOT AN EXTRA

Every performance number must come with machine, compiler, flags, problem size, repetitions and statistic. A number without those is an anecdote, and the marks in Parts B and C are for the reasoning that a bare number cannot support.

Policies

Late: 80 percent within 48 hours, then not accepted without documented cause.

Collaboration: discuss freely, write your own code and your own report. All submissions are compared for similarity, with variable renaming and reformatting ignored.

AI tools: allowed for learning and for explaining errors. You must be able to explain every line and every number you submit. Where a submission and a viva disagree, the viva decides.